

DATA STRUCTURES

Introduction

1

General Rules & Regulations

- No Late Arrivals.
- No Disturbance In Class
- No Use Of Mobile Phones(keep Them Silent And In Your Pockets)
- No Retake Of Quizzes/Assignments
- No Late Submissions
- Student Is Responsible For His / Her Short Attendance.

Dishonesty and Plagiarism

- Any kind of cheating will be considered serious offense
- All parties involve in cheating will get **ZERO** Score
 - Exams, quizzes, assignments, projects
 - Single assignment plagiarism case= zero in assignment
 - Multiple assignments plagiarism cases= complete assignment category zero
 - Usage of any AI tool/ plagiarism threshold =**20% (nonnegotiable)**
- Habitual cases will be awarded "F"

Course Information

- Pre-requisite
 - Programming Fundamentals/Object Oriented Programming
- Course meeting times
 - Lectures: 2 Sessions/Week
- Course Resource
 - Google Classroom – you will receive an invite

Grading policy: Absolute

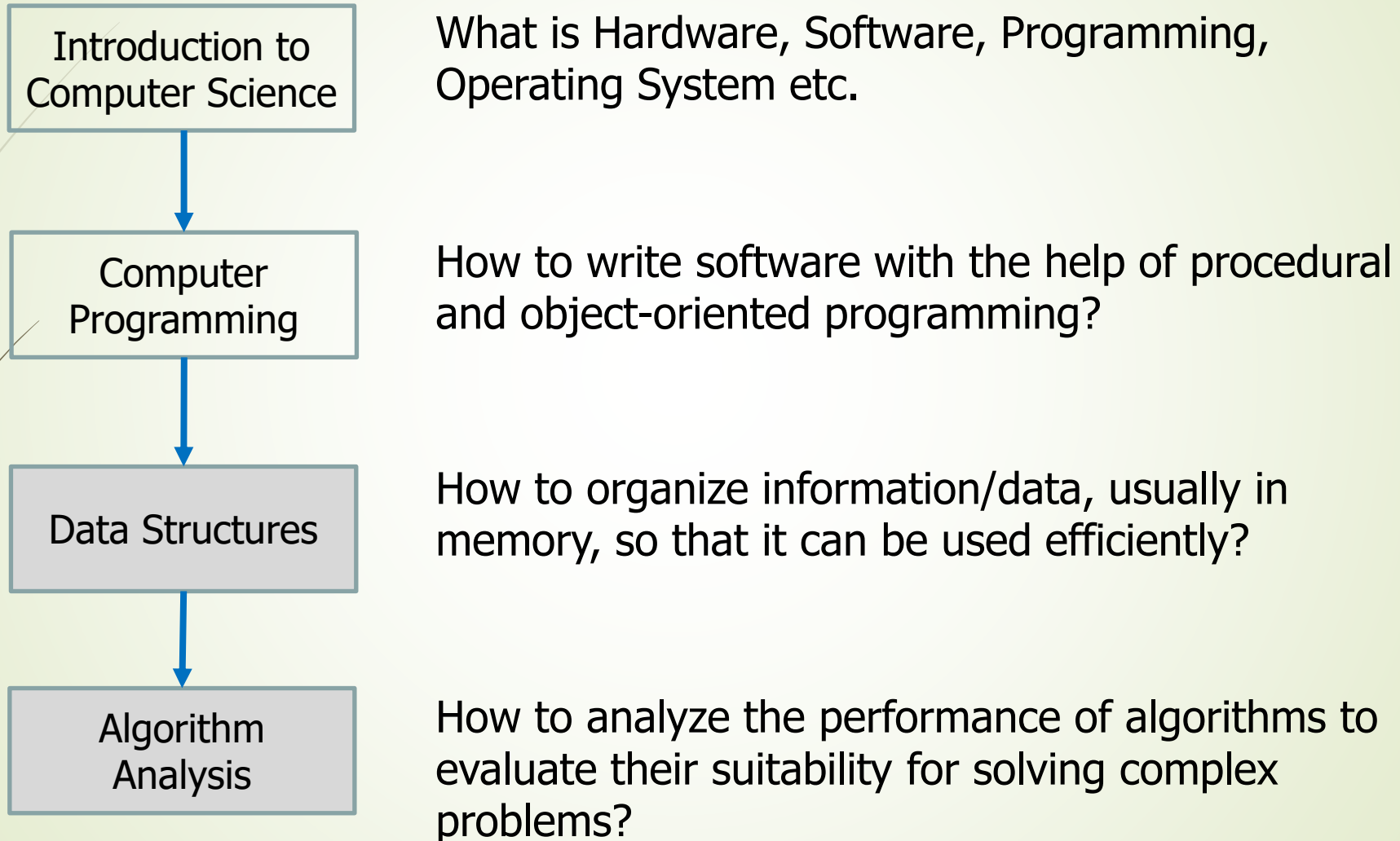
5

Assessment Type	Weight
Assignments (3)	10
Quizzes (≥ 5)	15
Sessional 1	12.5
Sessional 2	12.5
Project	10
Final Exam	40

Course & Reference Books

- Data Structures Using C++ (By D.S. Malik)
- Data Structures Using C and C++ (By Y. Langsam, M. J. Augenstein, A. M. Tenenbaum)
- Data Structures and Algorithms (By A. V. Aho, J. E. Hopcroft, J. D. Ullman)
- Schaum's Outline Series, Theory and problems of Data Structures (By Seymour Lipschutz)

General Overview



Introduction

- What is data?
 - information converted into binary/digital form
- Types of data?
 - text, numeric, audio, video etc.
- Importance of data?
 - primary purpose of computer programs is to store and retrieve information, usually as fast as possible

What is a Data Structure?

- In a general sense, any representation that is used for storing information is a data structure
 - Example: integers, structures, classes, arrays, etc.
- More typically, a data structure provides a way for organizing a collection of data items
 - Storing, accessing and modifying data items
- Most data structures have associated algorithms to perform operations on data
 - Search, insert, remove etc.

Costs and Benefits

- Each data structure requires
 - Space for storing data items
 - Time to perform each basic operation
 - Programming efforts
- Cost of a data structure
 - Time and space resources in consumes
- Choice of data structure depends on many factors
 - Type of data
 - Frequency with which various data operations are applied
- Hard to define data structure that performs better in all situations
 - Time and space tradeoff

Course Aim

- ▶ Learn the commonly used data structures
 - ▶ Form a programmer's basic data structure “toolkit”
- ▶ Case studies of data structures
- ▶ Examine the costs and benefits of every data structure

Motivational Example

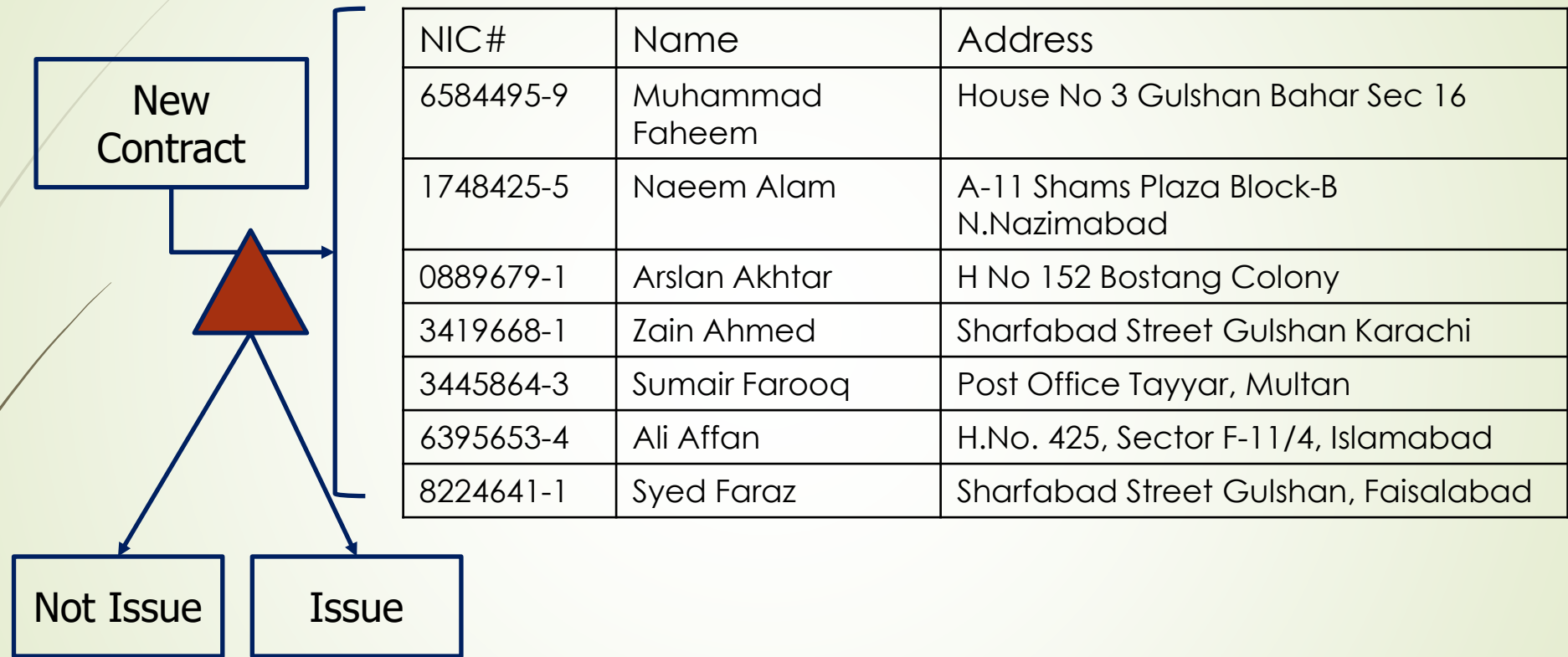
- A cellular service company provides contract to its 10 million users
- Due to new security enforcements, the company wants to **prevent issuing of multiple contracts to users**
- Method of Detecting Multiple Contracts
 - ❖ Before issuing a new contract to user
 - ❖ First search the **id of user** in existing contracts database
 - ❖ In case of failure → issue a new contract
 - ❖ In case of success → do not issue a new contract to user

Example: Linear Array Data Structure (1)

NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

- Linear Array (with 10 million entries)
 - ➔ 3 arrays (NIC, Name, Address)
 - ➔ Structured array
 - ➔ Class's object array

Example: Linear Array Data Structure (2)



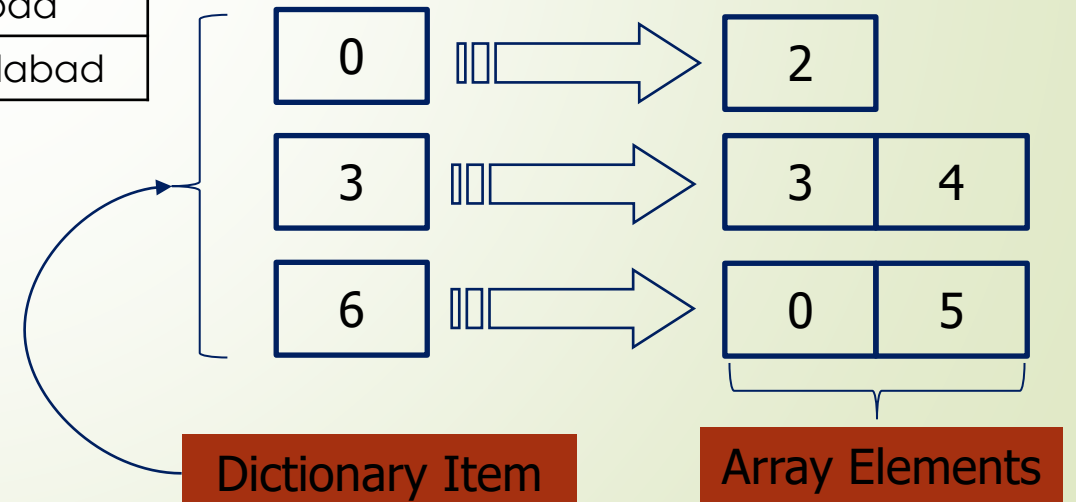
- Any disadvantage of linear array data structure?
- How to improve?

Example: Improved Data Structure

- Create a dictionary data structure
 - ➔ Group similar records together
 - ➔ Similarity in terms of first digit of NIC number
 - ➔ Add a dictionary entry for each distinct digit (0 – 9)
- Example: 3419668-1, 3445864-3, 1748425-5.
 - ➔ 3 and 1 are dictionary entries
- Existing contracts are searched in two steps
 1. Search the dictionary entry (i.e., group searching)
 2. Search contract within the group (i.e., with the same NIC digit)
- How much improvement w.r.t. linear array?

Example: Dictionary Data Structure (1)

NIC#	Name	Address
6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
0889679-1	Arslan Akhtar	H No 152 Bostang Colony
3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
3445864-3	Sumair Farooq	Post Office Tayyar, Multan
6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad



Example: Dictionary Data Structure (2)

- Another possibility
 - ➔ Maintain pointers with structures (or records)
 - ➔ Non-NULL pointer indicates presence of next record

	NIC#	Name	Address
0	6584495-9	Muhammad Faheem	House No 3 Gulshan Bahar Sec 16
3	1748425-5	Naeem Alam	A-11 Shams Plaza Block-B N.Nazimabad
6	0889679-1	Arslan Akhtar	H No 152 Bostang Colony
	3419668-1	Zain Ahmed	Sharfabad Street Gulshan Karachi
	3445864-3	Sumair Farooq	Post Office Tayyar, Multan
	6395653-4	Ali Affan	H.No. 425, Sector F-11/4, Islamabad
	8224641-1	Syed Faraz	Sharfabad Street Gulshan, Faisalabad

Data Structure

What is a data structure?

- In a general sense, any representation that is used for storing information is a data structure
- Example: An integer, structures, classes, arrays, etc.
- More typically, a **data structure** provides a **way for organizing a collection of data items**

Where data structure is helpful?

- Data structure can make the difference between a program running in a few seconds or many days
 - ➔ Efficiency of data structure

What is Data Structure Efficiency?

- Two major measures of efficiency of a data structure or algorithm
 - ➔ Time and space
- Cost of data structure
 - ➔ Time and space resources it consumes
- Efficiency of a data structure depends on its operations
 - ➔ Makes a difference between program running in few seconds or days
- Difficult to define data structure that performs better in all situations

Abstract Data Types

- A definition of data type solely in terms of
 - Set of related data items (or values)
 - Set of operations on the data
- Separation of logical properties from the implementation details
 - Hide implementation details (**Encapsulation!**)
- What not how is focus

Data Modeling

- Data Model is an abstract model that organizes elements of data and standardizes how they relate to one another and to the properties of real-world entities
 - explicitly determines the structure of data and therefore referred to as data structures in the context of programming languages
- The choice of particular data model depends on two consideration:
 - It must be rich enough in structure to represent the relationship between data elements
 - The structure should be simple enough that one can effectively process the data when necessary

Abstract Data Type (ADT)

Abstract Data Type

- *ADT* stores data and allow various operations on the data to access and change it
- A mathematical model, together with various operations defined on the model
- An ADT is a collection of data and associated operations for manipulating that data

Data Structures

- Physical implementation of an ADT
- Data structures used in implementations are provided in a language (*primitive* or *built-in*) or are built from the language constructs (*user-defined*)
- Each operation associated with the ADT is implemented by one or more subroutines in the implementation

Abstract Data Type (ADT)

- ADT is a way of looking at a data structure focusing on what it does and ignoring how it does its job
 - E.g., A **stack** is an abstract data type supporting *push*, *pop* and *isEmpty* operations
 - It can be implemented using an array or using a linked-list
- ADTs support *abstraction*, *encapsulation*, and *information hiding*
 - *Abstraction* is the structuring of a problem into well-defined entities by defining their data and operations
 - The principle of hiding the used data structure and to only provide a well-defined interface is known as *encapsulation*

Abstract Data Type (ADT)

- The core operations of ADT
 - Every Collection ADT should provide a way to:
 - add an item
 - remove an item
 - find, retrieve, or access an item
 - Many, many more possibilities
 - is the collection empty
 - make the collection empty
 - give me a sub-set of the collection

Example: Airplane Flight Reservation (1)

- Consider example of an airplane flight with 10 seats to be assigned
- Operations
 - ➔ List available seats
 - ➔ Reserve a seat
- Implementation: How to store, access data?
 - ➔ 10 individual variables



Implementation: 10 Individual Variables

List available seats:

1. if seat1 == ' ';
 display 1
2. if seat2 == ' ';
 display 2
- .
- .
- .
10. if seat10 == ' ';
 display 10

Reserve a seat:

1. Set DONE to false
2. if seat1 == ' ';
 print "do you want seat #1??"
 Get answer
 if answer=='Y';
 set seat1 to 'X'
 set Done to True
3. if seat2 == ' ' and Done == false;
 print "do you want seat #2??"
 Get answer
 if answer=='Y';
 set seat2 to 'X'
 set Done to True
- .
- .
- .

Example: Airplane Flight Reservation (2)

- Consider example of an airplane flight with 10 seats to be assigned
- Operations
 - ➔ List available seats
 - ➔ Reserve a seat
- Implementation: How to store, access data?
 - ➔ 10 individual variables
 - ➔ [An array of variables](#)



Implementation: An array of variables

List available seats:

```
for number ranging from 0 to max_seats-1, do:  
    if seat[number] == ' '  
        Display number
```

Reserve a seat:

Reading number of seat to be reserved

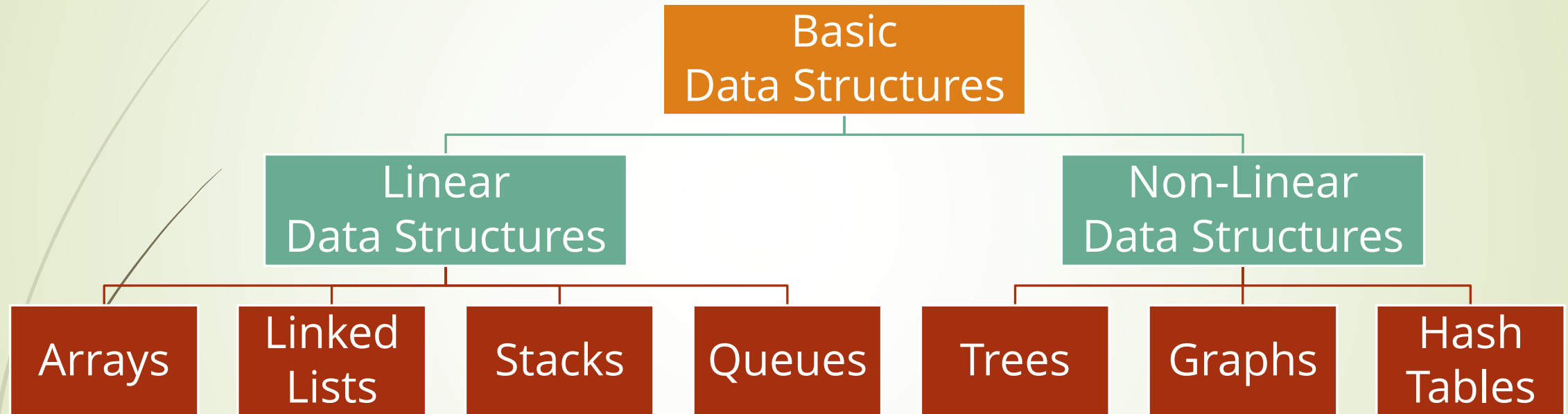
```
if seat[number] is equal to ' '  
    set seat[number] to 'X'  
else  
    Display a message that the seat having this number is  
    occupied
```



Example: Airplane Flight Reservation (2)

- This simple example illustrates the concept of an Abstract Data Type
- ADT consists of
 - ➔ Collection of data items
 - ➔ Basic operations that must be performed on them
- In the example, a collection of data is a list of seats
- Basic operations are
 - ➔ List available seats
 - ➔ Reserve a seat

Basic Data Structures



Basic Data Structures

➤ Linear: *Values are arranged in a linear fashion*

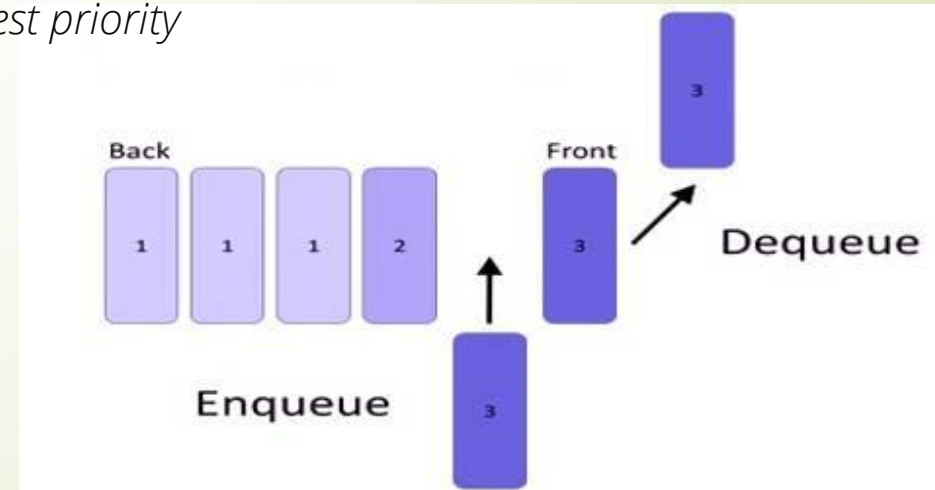
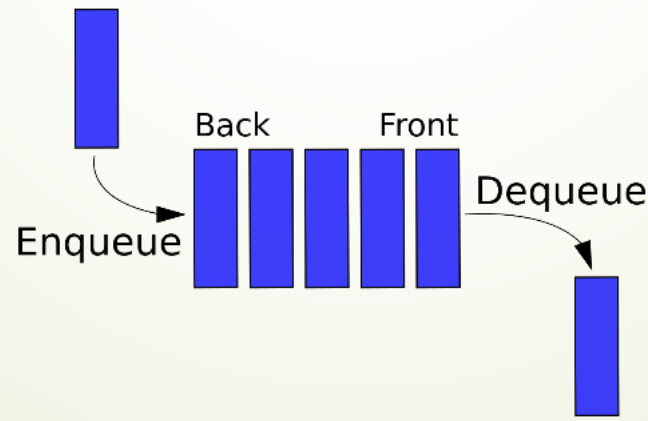
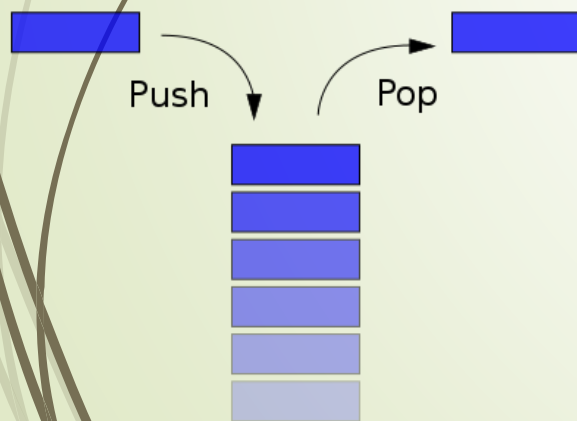
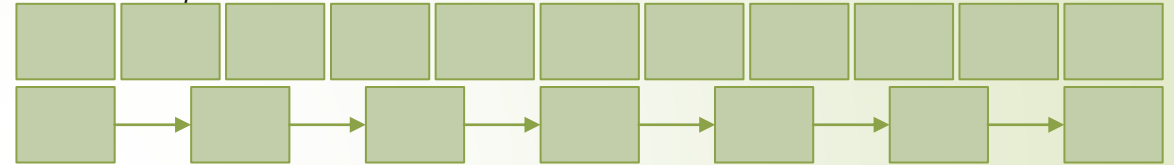
➤ Array: *Fixed size*

➤ Linked-List: *Variable size*

➤ Stack: *Add to top & remove from top*

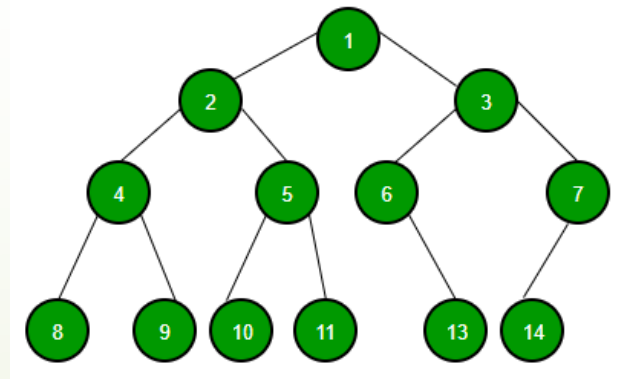
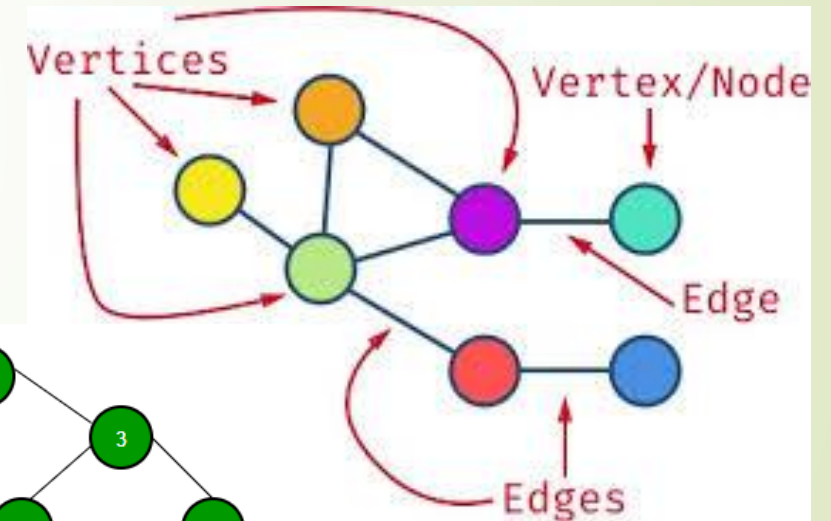
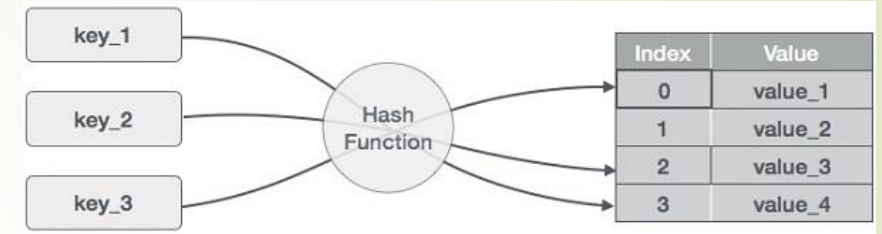
➤ Queue: *Add to back & remove from front*

➤ Priority Queue: *Add anywhere, remove the highest priority*



Basic Data Structures

- **Non-Linear:** *Values are not arranged in a linear fashion*
 - **Hash Table:** *Unordered lists which use hash function to insert & search*
 - **Tree:** *Data is organized in branches*
 - **Graph:** *A more general branching structure with less strict connection conditions than that for a tree*



Types of Data Structures (composition)

- **Homogenous:** *In this type of data structures, values of the same types of data are stored*
 - Array
- **Non-Homogenous:** *In this type of data structures, data values of different types are grouped and stored*
 - Structures
 - Classes

Index	1	2	3	4	5	6
Value	15	17	25	90	110	221

One - Dimensional Array

Index	1	2	3
1	10	15	7
2	9	25	30
3	39	2	84

Two - Dimensional Array

Index	1	2	3
1	10	15	7
2	9	25	30
3	39	2	84

Multi - Dimensional Array

Index	1	2	3
1	44	55	63
2	31	33	90



Selection of Data Structure

- Each data structure has costs and benefits
 - Rarely is one data structure better than another in all situations
- A data structure requires:
 - space for each data item it stores
 - time to perform each basic operation
 - programming effort
- Each problem has constraints on available time and space
 - Only after a careful analysis of problem characteristics can we know the best data structure for the task